ASSISTANT • 技术分享

文件检索与治理模块

已完成的能力 · 实测暴露的问题 · Phase A–E 路线图

v0.3.0 收尾 → v0.4.0 文件子系统 Phase A

2026-09-24 | 依据 docs/architecture/ 三份设计稿

本次范围

- 已完成：检索链路 + A1 止血
- 实测：索引其实只覆盖 Desktop
- 问题清单：P0 ×6 / P1 ×6 / P2 ×7
- 目标架构：四层 + 反馈回路
- 路线图：Phase A → B → C → D → E

AGENDA

这份 PPT 讲四件事

01

现在能做什么

检索链路、五级模糊匹配、
A1 止血的四项修复与实测数据

02

实测暴露了什么

P0/P1/P2 问题清单；
以及「索引其实只覆盖
Desktop」这个真问题

03

要建成什么样

四层架构、关键接口、
以及四个关键设计决策的取舍
理由

04

分几步走

Phase A（搜索基建）→ B（治
理过滤）
→ C/D/E（Box / 追踪 / 自动化
）

所有数字均来自本机实测或设计稿原文，未做估算加工。

模块定位

检索面与采集面：先把职责切开

文件检索（检索面）

让用户「找到想要的」：查询解析 → 候选筛选 → 打分 → top-K

只读：内存快照 + COW 发布，查询路径不走 SQLite

对应 Phase A（A1–A8）

文件治理（采集面）

保证「不漏文件」：种类过滤、工程排除、下载中转、去向追踪

写：磁盘遍历全量入库，只受排除规则约束

对应 Phase B–E

核心设计决定：两种「过滤」必须分清

磁盘遍历全量入库（只受排除规则约束）→**内存快照按当前白名单筛选构建**。于是切换「只要文档 / 含代码 / 全都要」只需**从 DB 重建快照（数十 ms），不必重新扫盘**。

分界线是 **files 表**：排除规则（L2/L3）决定**什么进 DB**；种类白名单只决定**内存快照装什么**（可随时切换）。

唯一例外：下载 box 不服从白名单 —— 否则白名单设成「只要 PDF」，刚下载的 .exe 会从收件箱消失，属数据丢失级事故。

已完成 ①

检索链路：Phase 2 已经跑通的一条路



三路候选 + 计算器，统一按 score 排序

来源	实现	进入条件	结果优先级
内置命令 / URL	AppBridge::m_allItems（10 条）	始终参与	priority 6–10
已索引应用	AppIndexer 扫描开始菜单 .lnk（本机 31 个）	始终参与	priority 5
文件	FileSearcher 全量索引 + FuzzyMatcher	仅非空查询，最多 20 条	priority 1
计算器	Calculator 递归下降解析	含数字且含运算符、长度 ≥3	score 1001 强制置顶

落点：include/core/FileSearcher.h · src/ui/AppBridge.cpp · resources/qml/MainWindow.qml

已完成 ②

FuzzyMatcher：五级打分 + 一条排序规则

分值	层级	含义	例子（查询 → 命中）
1000	完全匹配	大小写不敏感的全等	cmd → cmd
900	前缀匹配	以查询开头	repo → report.docx
800	词边界匹配	空格 / 连字符 / 下划线 / 点之后	cfg → app_cfg.ini
600–400	包含匹配	子串命中，位置越靠前分越高	port → report.docx
400–0	编辑距离	Levenshtein（仅短串启用）	reprot → report
0	不匹配	不进入结果集	—

设计取舍

· 五级语义保留为 tier 快速判定，A4 的 DP 打分在它之后细化。

· 文件 priority = 1：同分时排在应用与内置命令之后。

· 文件只在非空查询出现，空查询不返回文件。

· A4 拟补子序列匹配：chr → chrome_installer.exe。

排序规则（现状）

① score 降序（模糊分越高越靠前）；**② 同分按 priority 降序**（内置命令 > 应用 > 文件）；**③ 计算器结果以 score = 1001 prepend**，恒排第一。

P1-3 的问题：现在是**「收集全部命中 → std::sort 全排序 → erase 截断」**，而只需要 top-K；20 万条量级下 O(M log M) + 大数组拷贝会被放大。

已完成 ③

A1 止血：独立 patch，2026-09-24 落地

分支 fix/a1-stop-bleeding　已验证　4 个文件、104 行新增；旧 FileSearcher 接口与语义不变，单 commit 可 revert

缺陷	原计划做法	实际做法	结果
P0-1 跟随 reparse point	QDirIterator 加 QDir::NoSymLinks	实测无效 → 自控前序 DFS + isSymLink() || isJunction()	126 条 63 层叠罗汉路径 → 0
P0-2 热路径字符串分配	删掉 toNativeSeparators()	目录名命中即整棵子树不进 （真的剪枝，不只是省分配）	本机剪掉 160 个目录
P1-5 搜索写库时机	从 search() 挪到 executeCommand()	只在「真的执行了」时写； 补 debug 日志可观测	历史表不再被前缀垃圾污染
P1-6 输入防抖 + IME	Timer 40ms + inputMethodComposing	同计划	8 字符 → 1 次全量检索

⚠ **最重要的一条：计划里的 QDir::NoSymLinks 是个空操作 —— 编译通过、日志正常、看起来更好了，只有把索引 dump 出来才发现没生效。**

已完成 ③ • 验收

A1 三条验收：全部实测，不是推演

#	验收项	方法	结果
①	junction 不递归、不重复索引	临时 env 门控仪器把索引全路径落盘 + grep	含 /loop/ 的条目 126 → 0；探针文件恰好 2 条
②	连打 8 字符只触发 1 次 search	PostMessage 投 WM_CHAR ×8，按 debug 日志统计	只出现 1 行 'abcdefgh'，不是 8 行前缀
③	打字过程中 history.db 不新增行	打字前/后各查一次 count(*)	43 → 43 不变
③+	正向对照：执行时才写	输 1+2 后回车（计算器恒排第一，副作用仅写剪贴板）	43 → 44，新增行 query='1+2'

顺手做了两件事

清空 search_history 旧数据：44 → 0（清空前正是 1+ / 5e / isa / ope / de / del 这类前缀垃圾）；command_history 与 app_launch_history 未动。

验收手法沉淀成可复用脚本（out/a1_typing_probe.ps1）：走 Qt 真实消息链，不注入代码、不改被测程序。

顺带量到的两个数

索引建库 ≈ 900 ms（修前同一台机 713–2379 ms），剪枝 160 个目录。

小写化比较走 indexOf(..., CaseInsensitive)，全程零临时字符串 —— 目录级剪枝让「省分配」变成了「少枚举」。

P0：正确性与稳定性（6 条）

编号	问题	后果
P0-1	QDirIterator 默认过滤器含符号链接，Subdirectories 跟随 junction 递归	重复索引甚至无限递归 ✅ A1 已修
P0-2	跳过判定逐文件做 toNativeSeparators() + 5 次 contains	最热内循环里的设计错误 ✅ A1 已修
P0-3	20000 上限按遍历顺序任意截断，顺序取决于目录项返回顺序	结果不可解释、两次启动内容可能不同、D 盘永远进不来 ← 本机已实际发生
P0-4	索引在主线程、engine.load() 之前同步构建	冷启动白屏数百 ms ~ 数秒
P0-5	索引建完即冻结，无 watcher、无手动刷新入口	新建/改名/删除的文件在本次运行中永远不反映
P0-6	namespace + 文件级静态 s_index，无实例无注入点	不可单测 / mock / 多实例，与项目其他模块风格不一致

✅ 标记为本次 A1 已修复；P0-3/P0-4/P0-5/P0-6 分别由 A3 / A2 / A5 / A2 承接。

性能债与检索质量缺口

P1 — 性能

编号	问题
P1-1	每按键全量线性扫 2 万条 + 完整五级打分（O(N·L)，约 5–15 ms，未实测）
P1-2	FuzzyMatcher 内反复 toLower()，每次分配新 QString → N 次堆分配/按键
P1-3	收集全部命中后全排序再截断，而只需要 top-K
P1-4	QString(UTF-16) 同时存 name / path，目录串高度重复 → 2 万条约数 MB
P1-5	每按键同步写 search_history ✅ A1 已修
P1-6	QML onTextChanged 无防抖 ✅ A1 已修

P2 — 检索质量 / 体验

编号	问题
P2-1	只匹配文件名，路径不参与；无空格分词
P2-2	无子序列匹配：chr 命中不到 chrome_installer.exe
P2-3	无 frecency / 使用频率权重
P2-4	空查询时文件完全不出现（应给「最近文件」）
P2-5	IME 预编辑触发检索；中文无法用拼音首字母命中
P2-6	无扩展名独立权重，docx 命中大量噪音
P2-7	结果动作单一：缺 Ctrl+Enter 打开所在文件夹、复制路径

另有 P3 能力缺口：索引无持久化（冷启动从零爬盘）、无可配置根目录与排除规则、无索引健康度可观测。

实测暴露的真问题

索引其实「名不副实」：用户搜不到自己的文档

35607

Desktop 文件数

20000

索引上限

0

Documents 进入索引的文件数

链式因果（P0-3 的具体后果）

roots 顺序
Desktop 第一

root[0]

▶

Desktop 单根
就吃满 2 万

35607 > 20000

▶

遍历提前 break

Documents 未开始

▶

Documents / Downloads
一个文件都没进

1652 / 596 全部落空

这意味着功能「看起来有、实际没有」：日志显示 indexed 20000 files 一切正常，但用户搜 Documents 里的东西永远搜不到。

结论：P0-3 的实际优先级高于原判断（实施清单原计划留给 A3）。修复方向已定：**取消按 frecency 驱逐**，改为**「完整索引优先 + 软预算告警 + 硬上限停止」**—— 对搜索而言，正确性远比内存重要，绝不能出现「文件凭空消失」。

目标架构

四层 + 一条反馈回路

采集层 Indexer

`Scanner + Store`

Win32 迭代式 DFS，目录级剪枝；产出不可变 COW 快照

查询层 QueryEngine

`Parser → Filter → Score → TopK`

归一化 → 位图预筛 → 分层打分 waterfall → top-K 最小堆

融合层 ResultMerger

`ISearchProvider k-way merge`

各 provider 取 top-K 后归并，再叠加 frecency 综合分

反馈层 Frecency

只在「执行」时写

异步写库、不重建快照；空查询返回 frecency top-8

硬约束：查询层只能通过 FileSnapshot 的只读原语访问数据 —— 物理表示（v1 的 vector<FileEntry>、后续的 SoA / 字典表）必须藏在快照接口之后，替换表示不改调用方。

关键接口

门面只做四件事，快照只给只读原语

```
// FileIndex — 门面（薄）
void start();
void refresh();
QList<Match> query(const Query&);
IndexStats stats() const;
void upsert(const QString& path);
void remove(const QString& path);

// 私有成员：Scanner / Store /
// SnapshotManager / QueryEngine
```

判定标准：新逻辑能不能放进四个协作者之一？能，就不要加到门面上。

```
// FileSnapshot — 唯一的查询表示
int     size() const;
QString nameAt(int i) const;
QString lowerAt(int i) const;
QString pathAt(int i) const;
qint64  sizeAt(int i) const;
qint64  mtimeAt(int i) const;
```

v1 物理表示是 std::vector<FileEntry>（20 万条 ≈ 50–70 MB）；SoA / 字典表 / UTF-8 存储都是 gated 优化，先有 baseline 再决定。

```
// ISearchProvider — A7 的稳定点，插件系统（GUIDE Phase 4）就挂在它上面
class ISearchProvider {
    virtual QString providerId() const = 0;
    virtual QList<SearchResult> query(std::string_view text, int limit) = 0;
};
```

关键决策 ①

为什么不用 SQLite FTS5 做文件名检索

理由	依据
本机根本不可用	vcpkg sqlite3 3.53.3 只启用 json1；扫描 sqlite3.dll 未命中 FTS5/FTS3/rtree。要用得先 vcpkg install "sqlite3[fts5]"，可能触发 qtbase 重配
trigram 解决不了主场景	只解决子串，不解决子序列（chr → chrome）与 typo —— 而那才是启动器的核心卖点
选择率太低	2–3 字符查询的 postings 接近全量，却要多付一份索引体积
仍要回表排序	拿到 rowid 后还得回表取 path / frecency 再排序，省不掉关键路径

结论：走「内存快照索引 + SQLite 持久化」路线 —— 查询路径完全不碰 SQLite，SQLite 只负责**冷启动加速与 frecency 存储**。内容全文检索另立项，不在本次范围。

同样被否掉的：Xapian / Tantivy(Rust) / Lucene（定位文档级全文、引外部运行时，与学习项目不符）；RocksDB / LevelDB（KV 不解决匹配算法，20 万级用不上 LSM）；自建 trigram 倒排（内存与快照方案收益重叠）。

关键决策 ②

并发、失效与「绝不静默驱逐」

COW 快照发布

构建完一次性发布不可变快照，读者整轮查询免锁。

为什么不用 QReadWriteLock + 原地修改：原地改会迫使查询持锁、重建期阻塞 UI，正好和修 P0-4（冷启动白屏）冲突。

优先 std::atomic<std::shared_ptr>（C++20），编译不过降级 std::shared_mutex。A2 先花 10 分钟验证 MSVC 是否支持。

不用 QFileSystemWatcher

Windows 后端基于 FindFirstChangeNotification，单句柄不递归子树。

对 6 个根下成千上万子目录会退化成逐目录注册句柄 —— 句柄爆炸，且新增子目录还要重注册。

改为 ReadDirectoryChangesW（bWatchSubtree=TRUE），每根一句柄。

溢出（ERROR_NOTIFY_ENUM_DIR）→ 发 Overflow 事件 → 该根全量重扫，不静默丢事件。

绝不静默驱逐

早期「索引满按 frecency 驱逐」的设计已明确废弃。

被驱逐后用户搜不到自己存在的文件 —— 行为变成「文件凭空消失」。

对搜索而言正确性远比内存重要。

改为三档显式可见：软预算 30 万条告警 / 硬上限 100 万条停止并提示；缩小规模靠 roots 与排除规则「少收」，而不是事后丢。

最易踩的坑：细化缓存必须存「上一轮全部命中」

缓存里存 top-K，会出现**「结果凭空消失」**的 bug：用户删掉一个字符后，本该回到结果集的条目因为不在上一轮 top-K 里而永远不出现。规则：缓存深度 1 + 查询长度 ≤2 不吃缓存；且必须有「缓存命中路径与全量路径结果集逐条一致」的用例。

位图预筛、细化缓存、charMask、SoA、并行分片 —— **全部 gated**：A4 先出 baseline 数据，命中阈值才做。为没有实测支撑的 P50<3ms 去堆复杂算法，是最可能发生的浪费。

关键决策 ③

两个数据库的边界，与「迁移先于字段」

	file_index.db	downloads.db
性质	派生数据 —— 可从磁盘重建	权威数据 —— 磁盘上重建不出来
写入方	只有 FileIndexDatabase 可写	只有 DownloadsDatabase 可写
迁移失败	直接丢弃重建（删文件重扫）	必须备份、失败拒绝启动，绝不静默重建
表	dirs / files / roots / frecency	downloads / move_journal / tracking_events / download_rules / sources / app_meta

DatabaseMigrator（A3 必须先做出来）

- 版本号用原生 PRAGMA user_version，不额外建表。
- 只做前向迁移，每步一事务；禁止 DROP TABLE + 重建。
- 启动时序：打开 DB → 读 user_version → 逐版本迁移 → commit → 之后才允许其它访问。
- 第一个真实用例：治理的 cat_mask / flags / root_id / origin_id + idx_files_sizemtime。

差量写入：三方差集

- diskSet = 磁盘遍历 {(path,size,mtime)}；dbSet = SELECT path,size,mtime。
- added / removed / changed 三方差集（changed 按 size+mtime 判定）。
- 单事务 + prepared statement 批量，每 1000 行提交一次 —— 避免每行一次 fsync。
- 启动两阶段：先 loadFromDb() 灌满快照立即可查询，再后台差量校正。

路线图 • Phase A

搜索基建（v0.4.0）：A1–A8，每步独立可编译可验收

步	内容	交付 / 验收
A1 ✅	止血：junction / 目录级剪枝 / 写库时机 / 防抖+IME	已落地 2026-09-24；独立 patch，三条验收全过
A2	结构：FileIndex 门面 + Snapshot + SnapshotManager（COW）+ 后台扫描线程	立 fileIndex.engine = legacy|v2 开关，异常可一行配置退回；冷启动无可感白屏
A3	采集：FileScanner（Win32 DFS + 目录级剪枝）+ FileIndexDatabase + DatabaseMigrator + 差量事务	扫描吞吐进 baseline；roots 生效；迁移框架先于字段增补落地
A4	查询：QueryEngine（Parser→Filter→Score→TopK）+ bench_file_index.cpp	先出 baseline（N = 2/5/10/20 万），再决定要不要做优化候选
A5	增量：FileWatcher（ReadDirectoryChangesW）+ FileSystemEvent 分发 + 溢出降级重扫	新建/改名/删除 ≤100 ms 可在搜索里看到；溢出用例
A6	冷启动与反馈：DB 载入 → 快照发布 → 后台差量；frecency 回路；空查询「最近文件」	冷启动到可查询 < 300 ms（20 万条）；杀进程重启仍走 DB（日志可证）
A7	融合与收尾：ISearchProvider + k-way merge + 打分日志开关 + Ctrl+Enter/复制路径	删除 FileSearcher 兼容层与 engine 开关后仍能编译运行；结果无回归
A8	（可选）拼音首字母：enablePinyinInitials，默认关闭	开启后 bg 可命中「报告」；不进关键路径

分支节奏：A1 单独 patch → A2–A4 一个 feature 分支 → A5–A7 一个分支。A 阶段完成即是可用产品。

路线图 • Phase B–E

文件治理（v0.4.x → v0.5.x）

阶段	目标	主要交付	关键约束 / 验收
B v0.4.x	搜索 + 可控范围	FileClassifier（16 位 FileKind 掩码）+ ExcludeRuleSet / ProjectGuard + QueryParser + 快照 catMask/flags + 过滤 Chip UI	白名单切换 < 50 ms 且不扫盘； node_modules 工程条目下降 ≥95%
C v0.4.x–v0.5.x	Download Box 中转	DownloadsDatabase（pipe_state / track_state 双状态）+ IngestEngine（A+B 主判定 + 退避）+ FileIdentity + BrowserSource	下载完成 ≤3 s 可见；100 次端到端 0 误拉；同名 0 覆盖；100% 可撤销
D	去向追踪	FileId 同卷 → quickSig 跨卷 → 离线对账四级 → Unknown 兜底 UI；USN Journal 单独最后做	同卷指纹命中 ≥99%、误判 0； 移动链 ≥3 级正确；USN 默认关闭
E	自动治理与多源	归档规则（默认 suggest）+ dry-run + 查重（quickSig/fullSig）+ Todo/剪贴板联动 + IM 源适配器（QQ/微信/钉钉）	自动移动必走 move_journal； IM 源最后做，不静默接管陌生目录

B 必须先于 C：cat_mask/flags 就是 files 表 v1→v2 的第一个真实迁移用例，顺便把 DatabaseMigrator 立起来。

验收门槛

性能预算（三档口径）与正确性硬门槛

口径：Target / Budget / Fail —— 超过 Budget 就要解释，超过 Fail 必须停下来处理。

指标（N = 20 万条）	Target	Budget	Fail
查询 P50 / P99 —— 查询长 1–3	< 3 / 8 ms	< 6 / 15 ms	P99 > 25 ms
查询 P50 / P99 —— 查询长 4–8	< 2 / 5 ms	< 4 / 10 ms	P99 > 20 ms
快照内存	≤ 30 MB	≤ 40 MB	> 60 MB（触发优化候选）
冷启动到「可查询」	< 150 ms	< 300 ms	> 600 ms
首次全量扫描 20 万条（后台）	< 8 s	< 15 s	> 30 s
增量：watcher 事件 → 可查询	< 100 ms	< 300 ms	> 1 s

正确性硬门槛（不可让步）

- golden 用例全命中：子序列 / 多词 / typo / 中文 / 短路路径。
- 缓存命中路径与全量路径结果集逐条一致。
- 种类白名单切换 < 50 ms 且不扫盘。
- 浏览器下载 100 次：0 次误拉未完成文件。
- 去向追踪同卷指纹命中 ≥99%，误判 0。
- 同名冲突 0 覆盖，撤销成功率 100%。

注意：上表 Target 值是设计稿的预期，**目前尚无实测支撑**（20 万条 5–15 ms 也是估算）。所以 A4 的第一步是**只建 baseline、不设断言**：测 N = 2/5/10/20 万的查询分位、冷启动、全量扫描、快照内存，结果写进 docs/devlog/，再回头校准这张表。

最严重的风险，与「明确不做」的清单

风险（按严重度）

风险	缓解
误移用户文件（最严重）	默认 suggest 不自动移动；autoOrganize 默认关； 所有移动走 move_journal，先写 intent 再执行
去向误判（比「不知道」更糟）	多候选一律落 Unknown，绝不猜； 指纹/签名唯一命中才确认
索引 / box / 追踪三路重复处理同一变更	200 ms 合并窗口 + 路径去重； box 拉取完成后直接通知索引 upsert
长路径 >260 字符	全程 \\?\ 前缀 + manifest longPathAware； 本期至少保证不崩溃
OneDrive 占位 / 网络盘	只读元数据安全；mtime 频繁变靠 200 ms 去抖； 网络盘默认不入 roots

复杂度警告： ① 目录级剪枝必须先判父目录再下探，天然要求自建迭代栈 —— 这正是 A1/A2 必须先落地的原因；② 时序错（扫描未完就反查）会导致候选集为空、幽灵条目堆积。

明确不做

- × 不做全文检索（需另立项并启用 sqlite3[fts5]）
- × 不做下载加速 / 断点续传
- × 不做云盘双向同步 / 跨设备
- × 不接管「另存为」到任意目录
- × 不自动删除任何文件
- × 不猜测文件去向
- × 不做归档代理
- × 不做全局 EventBus（只加 FileSystemEvent + 极轻量分发器）

NEXT

下一步

现在

A1 已落地，等 review / PR

分支 fix/a1-stop-bleeding：1 个 fix commit + 1 个 docs commit；
已在隔离 worktree 单独编译 + 冒烟通过

接着

Phase 3 真机走查 → tag v0.3.0

大改之前先有一个已知良好、可回退的 tag；
顺带可插 1–2 天打包 spike（vcpkg Qt 部署 + NSIS）

然后

A2 结构：门面 + COW 快照 + 后台扫描

先验证 MSVC 是否支持 std::atomic<std::shared_ptr>（10 分钟，别跳过），
不支持就换 std::shared_mutex

之后

A3 采集 → A4 baseline → A5 增量 → A6 冷启动 → A7 融合

A4 的 baseline 数据是后面所有优化决策的依据

依据：docs/architecture/{文件检索重构方案, 文件治理功能设计, v0.4-实施清单}.md | docs/devlog/2026-09-24.md | docs/troubleshooting/qt-nosymlinks-挡不住junction.md